

📘 09차시 · SFT + LoRA 심화 교육자료

> 60분 실습형 | 확인일 2026-11-05 | 제작 선택 모델: Qwen/Qwen2.5-0.5B-Instruct

---

01. 표지 · 목표 · 60분 진행순서

09차시 | SFT + LoRA: 지도 미세조정과 저랭크 적응의 결합

선수학습: 04차시 LoRA·PEFT, 07차시 데이터 전처리, 08차시 SFTTrainer. 10차시는 QLoRA로 확장한다.

실습 목표: 대화형 JSONL 데이터를 검증하고, SFTTrainer의 peft_config에 LoraConfig를 연결하여 5스텝 학습 및 어댑터 저장 절차를 구현한다.

시간 배분: 0~10분 개념 복습 / 10~20분 구조 이해 / 20~30분 환경·데이터 점검 / 30~50분 실습 / 50~60분 결과·평가.

완료 기준: dry-run 통과, 학습 가능한 매개변수 비율 확인, 학습 로그와 검증 손실 구분, 어댑터 파일의 저장 목적 설명.

주의: 5스텝·5개 샘플은 교육용 동작 확인이며 품질 개선을 증명하는 데이터가 아니다. 모델 선정은 제작상의 선택이다.

---

02. 배경 · 선수지식 · 전체 학습과의 차이

SFT(Supervised Fine-Tuning, 지도 미세조정)는 입력에 대응하는 목표 출력을 이용하여 다음 토큰의 확률을 조정한다. 08차시는 전체 모델 매개변수를 업데이트하는 예제였다.

LoRA(Low-Rank Adaptation, 저랭크 적응)는 원본 가중치 W를 동결하고 작은 두 행렬 A, B의 곱으로 업데이트를 표현한다. 원본 가중치 자체를 모두 학습하지 않는다.

PEFT(Parameter-Efficient Fine-Tuning, 매개변수 효율적 미세조정)는 적은 학습 매개변수로 모델을 적응시키는 방법들의 집합이다. LoRA는 대표적인 PEFT 기법이다.

08차시 대비 변경: SFTTrainer의 peft_config 인자 추가, 학습률 2e-5에서 1e-4로 교육용 변경, 출력물은 전체 모델이 아닌 어댑터 중심.

07차시에서 만든 processed/train.jsonl과 validation.jsonl을 재사용할 수 있다. 단, 이번 배포물의 data/ 파일은 독립 실습용 최소 예제이며 07차시 전체 데이터를 대체하지 않는다.

06차시에서 배운 공정한 비교 원칙을 적용한다. 같은 질문, 같은 디코딩 조건, 동일한 기준 모델과 실제 학습된 어댑터를 사용해야 한다.

03. 핵심 원리 · 행렬 크기와 학습률

기본 선형층은 y = Wx로 나타낼 수 있다. LoRA 적용 후 y = Wx + (alpha/r)BAx로 계산한다. W는 고정하고 A, B만 업데이트한다.

W의 크기가 d_out × d_in일 때 전체 매개변수 수는 d_out × d_in, LoRA 추가 매개변수 수는 r × (d_in + d_out)이다. r은 랭크를 뜻한다.

예를 들어 d_in=1024, d_out=1024, r=8이라면 전체 1,048,576개에 비해 LoRA는 16,384개이다. 이는 단일 투영층의 이론 비교이며 전체 모델 비율과 다르다.

alpha/r은 LoRA 업데이트 크기를 조절한다. 이번 예제는 r=8, alpha=16이므로 배율은 2이다. dropout=0.05는 학습 중 일부 입력을 확률적으로 제외한다.

학습률 1e-4는 LoRA 예제용 설정이며 보편적 최적값이 아니다. 손실이 발산하면 낮추고, 데이터가 작으면 과적합을 특히 경계한다.

Qwen2.5의 q_proj와 v_proj를 대상으로 삼는다. 모델 아키텍처가 다르면 이 이름이 없을 수 있으므로 모듈명을 먼저 확인해야 한다.

04. 구조도 · 데이터와 모델의 흐름

07차시 JSONL → Dataset.from_list → SFTTrainer → 대화 템플릿/토큰화 → 기본 Qwen2.5 모델(동결) + q_proj/v_proj의 LoRA A/B(학습) → 손실·역전파 → 어댑터 저장.

기본 가중치 W는 추론과 역전파 계산에 참여하지만 옵티마이저로 업데이트하지 않는다. LoRA A/B의 기울기만 최적화된다.

훈련 경로와 검증 경로는 별개다. train_dataset은 가중치 갱신에 사용하고 eval_dataset은 일반화 상태를 측정한다.

저장된 어댑터를 재사용할 때는 학습 당시와 동일한 기본 모델 식별자 및 토크나이저 구성을 유지해야 한다.

구조 검증 포인트: 모델 ID, LoRA target_modules, train/eval 데이터, output_dir, 저장 파일의 의미가 코드와 일치해야 한다.

```
flowchart LR
 A["train.jsonl / validation.jsonl"] --> B["Dataset"]
 B --> C["SFTTrainer"]
 C --> D["Qwen2.5 기본 모델 – 동결"]
 C --> E["LoRA q_proj/v_proj - 학습"]
 E --> F["어댑터 저장"]
```

05. 준비 · 설치 · 실행환경

Ubuntu + Python 3.11 + VS Code 환경에서 가상환경을 만든다. GPU 학습에는 CUDA를 지원하는 PyTorch 설치가 필요하다.

터미널 명령: python3.11 -m venv .venv (프로젝트 전용 파이썬 환경 생성)

터미널 명령: source .venv/bin/activate (가상환경 활성화)

터미널 명령: python -m pip install --upgrade pip (패키지 설치 도구 갱신)

터미널 명령: python -m pip install torch transformers datasets trl peft accelerate (필수 라이브러리 설치; torch CUDA 빌드는 공식 선택표 확인)

터미널 명령: python -c "import torch; print(torch.cuda.is_available())" (CUDA 사용 가능 여부 확인)

터미널 명령: python -m pip freeze > requirements-resolved.txt (실습 당시 설치 버전 기록)

GPU 메모리 부족 시 max_length를 128로 줄이고 다른 GPU 프로세스를 종료한다. 10차시 양자화 학습과 혼동하지 않는다.

06. 실습 코드 · 주요 구문과 실행

실습 파일: lesson09_sft_lora.py. 전체 소스는 동일 폴더에 제공하며 Python 한글 주석을 포함한다.

python lesson09_sft_lora.py --dry-run (GPU 없이 데이터 형식과 중복 검사)

python lesson09_sft_lora.py (CUDA GPU가 있을 때 실제 5스텝 학습 수행)

LoraConfig(r=8, lora_alpha=16, target_modules=["q_proj", "v_proj"], task_type="CAUSAL_LM")은 저랭크 어댑터의 크기와 적용 위치를 지정한다.

SFTConfig(max_steps=5, max_length=256, gradient_accumulation_steps=2, eval_strategy="steps", eval_steps=5)는 짧은 학습과 5스텝 평가를 설정한다.

SFTTrainer(model=MODEL_ID, args=config, train_dataset=..., eval_dataset=..., processing_class=tokenizer, peft_config=lora)는 SFT와 LoRA를 연결하는 핵심 호출이다.

실행 전 터미널 작업 폴더를 09_SFT_LoRA로 설정해야 data/train.jsonl 상대경로가 유효하다.

전체 실습 코드

```
"""09차시: SFTTrainer + LoRA. --dry-run은 GPU 없이 데이터 형식만 검사."""
import argparse
import json
from pathlib import Path

MODEL_ID = "Qwen/Qwen2.5-0.5B-Instruct"  # 교육용 모델 선택


def read_data(path):
    """JSONL의 messages 구조와 최소 user/assistant 존재를 검증한다."""
    rows = []
    for n, line in enumerate(Path(path).read_text(encoding="utf-8").splitlines(), 1):
        if not line.strip():
            continue
        row = json.loads(line)
        msgs = row.get("messages")
        if not isinstance(msgs, list) or not msgs:
            raise ValueError(f"{path}:{n}: messages 누락")
        roles = []
        for msg in msgs:
            if not isinstance(msg, dict) or msg.get("role") not in {"system", "user", "assistant"}:
                raise ValueError(f"{path}:{n}: 역할 형식 오류")
            if not isinstance(msg.get("content"), str) or not msg["content"].strip():
                raise ValueError(f"{path}:{n}: 빈 내용")
            roles.append(msg["role"])
        if "user" not in roles or "assistant" not in roles:
            raise ValueError(f"{path}:{n}: user/assistant 쌍 필요")
        rows.append({"messages": msgs})
    if not rows:
        raise ValueError(f"{path}: 비어 있는 데이터")
    return rows


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--train", default="data/train.jsonl")
    p.add_argument("--validation", default="data/validation.jsonl")
    p.add_argument("--output", default="outputs/lesson09_lora")
    p.add_argument("--dry-run", action="store_true")
    args = p.parse_args()
    train, valid = read_data(args.train), read_data(args.validation)
   # 학습/검증 사이 동일 대화가 중복되지 않도록 원문을 비교한다.
    canonical = lambda x: json.dumps(x["messages"], ensure_ascii=False, sort_keys=True)
    overlap = {canonical(x) for x in train} & {canonical(x) for x in valid}
    if overlap:
        raise ValueError(f"학습/검증 데이터 중복: {len(overlap)}건")
    print(f"DATA PASS: train={len(train)}, validation={len(valid)}, overlap=0")
    if args.dry_run:
        print("DRY RUN PASS: 모델 다운로드 · GPU 학습 미실행")
        return

    import torch
    from datasets import Dataset
    from transformers import AutoTokenizer
    from peft import LoraConfig
    from trl import SFTConfig, SFTTrainer
```

```
    if not torch.cuda.is_available():
        raise RuntimeError("CUDA GPU가 필요합니다. 우선 --dry-run을 실행하세요.")
    tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token
    # Qwen2.5 attention 투영 계층만 저랭크 행렬로 학습한다.
    lora = LoraConfig(
        r=8, lora_alpha=16, lora_dropout=0.05,
        target_modules=["q_proj", "v_proj"],
        bias="none", task_type="CAUSAL_LM",
    )
    config = SFTConfig(
        output_dir=args.output, max_length=256,
        per_device_train_batch_size=1, gradient_accumulation_steps=2,
        max_steps=5, learning_rate=1e-4,
        logging_steps=1, eval_strategy="steps", eval_steps=5,
        save_strategy="no", report_to="none", fp16=True,
        seed=42, assistant_only_loss=False,
    )
    trainer = SFTTrainer(
        model=MODEL_ID, args=config,
        train_dataset=Dataset.from_list(train),
        eval_dataset=Dataset.from_list(valid),
        processing_class=tokenizer, peft_config=lora,
    )
    trainer.model.print_trainable_parameters()
    result = trainer.train()
    print("학습 손실:", result.training_loss)
    print("검증 결과:", trainer.evaluate())
    # 여기에는 전체 기본 모델이 아닌 LoRA 어댑터가 저장된다.
    trainer.save_model(args.output)
    tokenizer.save_pretrained(args.output)
    print("LoRA 어댑터 저장:", args.output)


if __name__ == "__main__":
    main()
```

07. 코드 해설 · 실제 검증과 예상 출력

read_data()는 JSONL을 읽어 messages의 role/content가 유효한지 확인한다. 학습과 검증의 완전히 동일한 대화 중복도 차단한다.

--dry-run은 torch, datasets, trl, peft를 불러오기 전에 종료한다. 따라서 데이터 구조는 검증하지만 API 호환성·GPU 학습은 검증하지 않는다.

실제 실행된 출력: DATA PASS: train=10, validation=2, overlap=0 / DRY RUN PASS: 모델 다운로드·GPU 학습 미실행.

GPU 실행 시 기대되는 출력 형식: trainable params: ... / train loss ... / eval_loss ... / LoRA 어댑터 저장: outputs/lesson09_lora. 수치는 GPU 환경에서 측정해야 하며 예시 값을 실제 결과로 간주하지 않는다.

print_trainable_parameters()는 동결/학습 매개변수 비율을 보여준다. 수치는 라이브러리 버전과 타깃 모듈에 따라 달라질 수 있다.

검증 손실 감소는 좋은 신호일 수 있지만, 1개 검증 샘플과 5스텝만으로 일반화 성능을 주장할 수 없다.

08. 따라하기 · 60분 실습 시나리오

1단계(5분): data/train.jsonl과 data/validation.jsonl의 messages 구조를 열어 user와 assistant가 들어 있는지 확인한다.

2단계(5분): python lesson09_sft_lora.py --dry-run 실행. 데이터 수와 overlap=0을 기록한다.

3단계(5분): GPU 확인. python -c "import torch; print(torch.cuda.is_available())" 실행.

4단계(10분): 코드에서 r=8, alpha=16, target_modules, learning_rate, max_steps를 찾고 각각의 기능을 설명한다.

5단계(15분): CUDA 환경에서 python lesson09_sft_lora.py 실행. 모델 다운로드와 학습 로그를 관찰한다.

6단계(10분): trainable params와 전체 params의 비율, 학습 손실, eval_loss, 저장 경로를 기록한다.

7단계(10분): 08차시와 비교해 추가된 peft_config와 저장물 차이를 설명하고, 학습하지 못한 경우 dry-run 결과와 미검증 사항을 보고한다.

---

09. 오류 대처 · 확장실습

FileNotFoundError: 실행 위치를 확인하거나 --train, --validation으로 정확한 경로를 지정한다.

CUDA 미사용: 드라이버, torch CUDA 빌드, GPU 지원 여부를 확인한다. CPU만으로 실제 학습을 수행했다고 표시하지 않는다.

CUDA out of memory: 다른 프로세스를 종료하고 max_length를 128로 줄이거나 배치·기울기 누적 조합을 조정한다.

Target modules not found: 해당 모델에 q_proj, v_proj가 있는지 named_modules()로 확인한다. 이름이 다르면 임의로 유지하지 않는다.

채팅 템플릿 오류: tokenizer.chat_template과 EOS 토큰을 확인한다. Qwen2.5는 <|im_end|> 종료 토큰 설정을 확인할 가치가 있다.

확장 A: r=4와 r=8을 비교하여 학습 매개변수 수 변화만 확인한다. 확장 B: 07차시 processed 데이터로 교체해 데이터 누출 검사를 다시 수행한다.

확장 C: 같은 검증 질문을 학습 전후로 비교하되 06차시 평가 원칙에 따라 실제 추론 로그를 남긴다.

10. 평가 · 체크리스트 · 공식 참고자료

확인 문제 1: SFTTrainer에 LoRA를 연결하는 인자는 무엇인가? 답: peft_config.

확인 문제 2: 기본 가중치 W와 LoRA A/B 중 어떤 것이 학습되는가? 답: 기본 W는 동결, A/B가 학습된다.

확인 문제 3: 어댑터만 저장했을 때 추론에 필요한 것은? 답: 동일한 기본 모델과 호환 토크나이저 및 어댑터.

실습 체크리스트: [ ] JSONL 형식 [ ] train/validation 분리 [ ] dry-run PASS [ ] GPU 상태 확인 [ ] 학습 가능한 비율 기록 [ ] 손실 해석 [ ] 저장 경로 확인.

공식 출처 확인일: 2026-11-05. TRL SFTTrainer https://huggingface.co/docs/trl/sft_trainer

PEFT LoraConfig https://huggingface.co/docs/peft/main/package_reference/lora

Qwen2.5-0.5B-Instruct https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct

검증 수준: Python AST/py_compile, 데이터 dry-run, PDF 10쪽·한글 추출 확인. 실제 GPU 학습 및 추론은 미검증.

다음 10차시: 4비트 양자화 기반 QLoRA를 도입하여 SFT+LoRA의 GPU 메모리 요구와 비교한다.

데이터 경로

`data/train.jsonl`, `data/validation.jsonl`은 동봉된 독립 예제이며, 07차시의 처리 데이터를 복사해 대체할 수 있습니다.

적대적 검증·개선 기록 (2026-11-05)

반영: 학습 기본 데이터 파일을 07차시의 10/2 분할로 통일했습니다. 기존 4/1 축소 예제는 optional_sample_data에 보존했습니다.

검증 구분: 코드 구문·입력 데이터·PDF 및 HTML 생성과 검증은 수행했으나 실제 모델 다운로드·GPU 학습·추론 품질·Ollama 배포는 별도의 실물 검증이 필요합니다.

재현성: 교육용 5스텝 학습의 손실 감소나 정확도 향상을 보장하지 않습니다. 모델 ID, 데이터 버전, 장치·패키지 버전, 난수 시드, 생성 설정을 기록해야 합니다.

공식 근거: SFTTrainer, PEFT 연동, 템플릿, PEFT 모델 저장, 양자화, Ollama GGUF (2026-11-05 확인)